

Question 1: Building an ETL Pipeline

Introduction

This report contains the development of an ETL pipeline for cryptocurrency data. The pipeline extracts data from two sources, transforms it into a structured format, and loads it into SQLite database for further analysis. This follows the fundamental ETL methodology covered in lectures, where data is collected from multiple sources, cleaned and stored in a standardized format.

Part A: Extraction

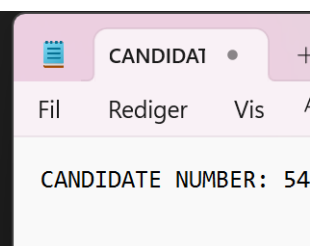
A1 - API Extraction:

The historical market data from the CoinGecko API was extracted for three assets: Bitcoin, Ethereum and Solana. The endpoint used was:

https://api.coingecko.com/api/v3/coins/{coin}/market_chart

Parameters that was included was `vs_currency=usd`, `days=14` and `interval=daily`. A reusable function `fetch_coin_data()` was created to fetch data for each coin and return a structured pandas DataFrame that contains `coin_id`, timestamp, price and volume. The three DataFrames was combined using `pandas.concat()`

	price	volume	coin_id	date
0	77425.715192	2.298513e+10	bitcoin	2026-05-18
1	76952.214859	4.360067e+10	bitcoin	2026-05-19
2	76808.810553	2.889639e+10	bitcoin	2026-05-20
3	77459.944346	2.863719e+10	bitcoin	2026-05-21
4	77546.337653	2.907525e+10	bitcoin	2026-05-22



A2 - Web Scraping:

Scraped data for the top 15 cryptocurrencies from CoinMarketCap using the request and BeautifulSoup libraries. It parsed the HTML response and located the table using `find_all("tr")`. It was required to have User-Agent header, because if else you would just get blocked by the server. These fields were extracted using CSS class selectors: name, symbol, price and 7d_change.

	name	symbol	price	7d_change
0	CoinMarketCap 20 Index DTF	CMC20	\$149.44	3.00%
1	Bitcoin	BTC	\$73,553.44	3.62%
2	Ethereum	ETH	\$2,008.42	4.07%
3	Tether	USDT	\$0.9985	0.01%
4	BNB	BNB	\$718.96	9.78%
5	XRP	XRP	\$1.32	1.35%
6	USDC	USDC	\$0.9996	0.01%
7	Solana	SOL	\$81.95	3.84%
8	TRON	TRX	\$0.3482	4.68%
9	Hyperliquid	HYPE	\$68.28	7.22%
10	Dogecoin	DOGE	\$0.09993	1.90%
11	UNUS SED LEO	LEO	\$10.01	0.21%
12	Zcash	ZEC	\$552.16	16.42%
13	Cardano	ADA	\$0.2341	3.56%
14	Stellar	XLM	\$0.2490	69.54%

Part B: Transformation & Feature Engineering

B1 – String Cleaning

The `7d_change` column from the scraped dataset contained percentage signs. The `%` symbol was removed using `str.replace()` and it was converted into a numeric float using `astype(float)`. The `price` column contained dollar signs and commas, that were removed before converting to float, and both columns were confirmed to be of type `float64` after cleaning.

name	object
symbol	object
price	object
7d_change	float64
dtype: object	
0	3.00
1	3.62
2	4.07
3	0.01
4	9.78
Name: 7d_change, dtype: float64	

B2 – Time-Series Formatting

The Unix timestamps returned by the CoinGecko API were in milliseconds. It was transformed to human readable dates in year, month and date format using `pandas.to_datetime()` with `unit='ms'` and `.dt.strftime('%Y-%m-%d')`.

The original timestamp column was then dropped from the dataframe.

	price	volume	coin_id	date
0	77425.715192	2.298513e+10	bitcoin	2026-05-18
1	76952.214859	4.360067e+10	bitcoin	2026-05-19
2	76808.810553	2.889639e+10	bitcoin	2026-05-20
3	77459.944346	2.863719e+10	bitcoin	2026-05-21
4	77546.337653	2.907525e+10	bitcoin	2026-05-22

This output is identical to the one that was shown in part A1, because the timestamp conversion was applied right after the data was loaded. Date column verified that the transformation was successful.

B3 – Rolling Average

I calculated a 3-day rolling average for each coin using `groupby("coin_id")` combined with `.rolling(window=3).mean()`. This will group the data by coin before calculating the moving average, which results in that Bitcoin's average wont bleed into Ethereum's data. The results of the NaN values for the first two rows of each coin were filled with the original price using `fillna()`. The name of the new column was `price_3d_mavg`.

	price	volume	coin_id	date	price_3d_mavg
0	77425.715192	2.298513e+10	bitcoin	2026-05-18	77425.715192
1	76952.214859	4.360067e+10	bitcoin	2026-05-19	76952.214859
2	76808.810553	2.889639e+10	bitcoin	2026-05-20	77062.246868
3	77459.944346	2.863719e+10	bitcoin	2026-05-21	77073.656586
4	77546.337653	2.907525e+10	bitcoin	2026-05-22	77271.697517
5	75482.523800	3.031402e+10	bitcoin	2026-05-23	76829.601933
6	76672.792891	3.302147e+10	bitcoin	2026-05-24	76567.218114
7	76988.161261	2.334675e+10	bitcoin	2026-05-25	76381.159317
8	77274.401522	2.157971e+10	bitcoin	2026-05-26	76978.451891
9	75824.063325	3.915969e+10	bitcoin	2026-05-27	76695.542036

Part C: Data Loading and Verification

C1 – Database Creation

I set up a local SQLite database called `crypto_etl.db` using Python's `sqlite3` library. I chose SQLite because it's simple to use and works well for these types of projects. I stored the transformed API data in a table called `historical_data`, and the cleaned scraped data was kept in a table called `current_data`. To insert the data into the database, I used pandas' `to_sql()` method with `if_exists="replace"`. This made it easier to rerun the script without worrying about duplicate tables.

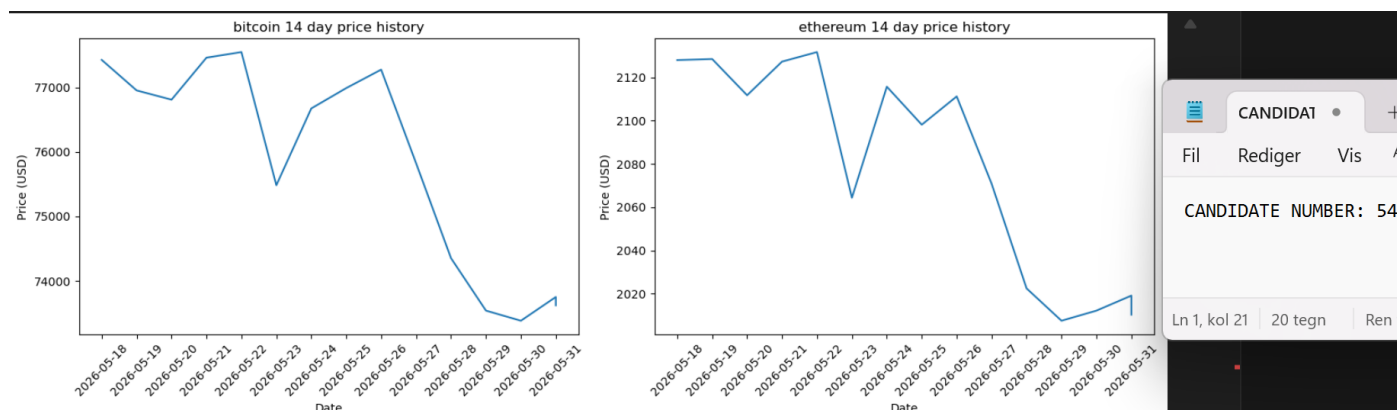
C2 – SQL Verification

I executed a SQL query using `pandas.read_sql_query()` to collect the top 5 rows from `historical_data` where `coin_id='solana'`. This query helped me confirm that the data was actually being stored correctly and could be queried without issues

	price	volume	coin_id	date	price_3d_mavg
0	85.158905	2.156422e+09	solana	2026-05-18	85.158905
1	85.253386	3.104441e+09	solana	2026-05-19	85.253386
2	84.267408	1.981280e+09	solana	2026-05-20	84.893233
3	86.005667	2.723522e+09	solana	2026-05-21	85.175487
4	87.149114	3.219780e+09	solana	2026-05-22	85.807397

Part D: Data Visualization

For visualization, I created a figure with two subplots using Matplotlib. The first plot shows Bitcoin's price over the last 14 days, while the second shows Ethereum's price over the same period. I placed them side by side to make it easier to compare trends. Both subplots contain axis labels, titles, and rotated x-axis date labels for readability. This chart shows a general downward price trend for both assets over the 14-date period.



Part E: Dataset Insights

E1 – Data Merging & Price Discrepancy

Bitcoin and Ethereum's most recent price was extracted from the `historical_data` table and merged with the `current_data` table from the scraped dataset. I calculated the absolute dollar differences between the two sources.

	coin_id	price_api	price_scraped	price_diff	CANDIDATE NUMBER: 54
0	bitcoin	73621.380247	73553.44	67.940247	
1	ethereum	2010.362204	2008.42	1.942204	

The results show a difference of 67.94 dollars for Bitcoin, and 1.94 dollars for Ethereum. These discrepancies are expected given that the two sources aggregate prices differently.

E2 – Trend Verification

The change for Bitcoin through the 7-day percentage was calculated manually using the API data by comparing the most recent price to the price exactly 7 days prior. The result was compared to the scraped `7d_change` value from CoinMarketCap.

Calculated 7d change: -4.37%	CANDIDATE NUMBER: 54
Scraped 7d change: 3.62%	

The change that was calculated shows -4.37% while CoinMarketCap reported +3.62%. This significant difference is addressed in the critical reflection below.

E3 – Critical Reflection

The 7-day change that was calculated from the CoinGecko API (-4.37%) contrasts considerably from the value reported on CoinMarketCap (+3.62%). This variance can be explained by several factors. First of all, the two platforms use different reference timestamps. CoinGecko aggregates daily price across the whole day, while CoinMarketCap may snapshot prices at a specific UTC time, which means that "7 days ago" refers to slightly different moments. Second of all, the time zone differences between data sources can shift which calendar day a price belongs to. Afterall the price aggregation methods differ, as CoinGecko

averages prices across several exchanges, while CoinMarketCap weights by trading volume, which results in different baseline prices for the same asset.

Conclusion

This ETL pipeline was successfully extracted and transformed. It loaded cryptocurrency data from two different sources into a structured SQLite database. The pipeline shows are how raw API and scraped data can be cleaned and combined for meaningful analysis.

Question 2: Comparative Data Analysis and Fare Prediction using NYC Yellow and Green Taxi

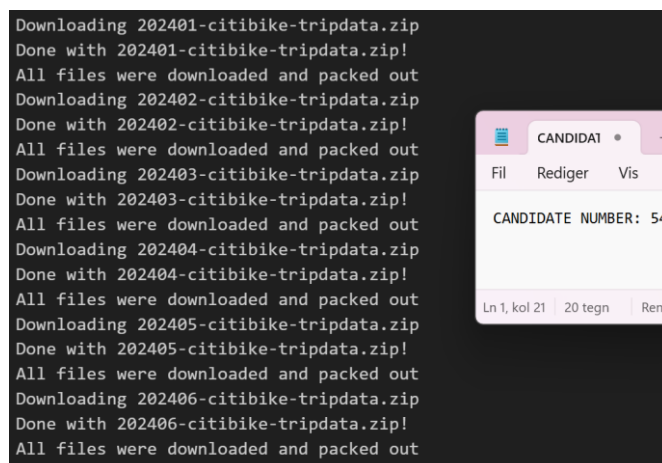
Introduction

This question contains the processing and analysis of NYC Citi Bike trip data. To do this I used PySpark for big data handling, because it consists of millions of bicycle-sharing records from January through June 2024, extracted from a public AWS S3 bucket.

I chose PySpark instead of pandas because the dataset was really big (about 3 GB) and it contained almost 19 million rows. This could cause memory issues in the regular pandas environment.

Part A: Programmatic Data Extraction

I used requests and zipfile libraries to download six monthly zipfiles programmatically. It covered January through June 2024. The naming format of the files was `2024{MM}-citibike-tripdata.zip` and it was extracted into a local folder named `citibike_data/`. The amount of total extracted data was about 3.43 GB that consisted of 22 CSV files.



```
Downloading 202401-citibike-tripdata.zip
Done with 202401-citibike-tripdata.zip!
All files were downloaded and packed out
Downloading 202402-citibike-tripdata.zip
Done with 202402-citibike-tripdata.zip!
All files were downloaded and packed out
Downloading 202403-citibike-tripdata.zip
Done with 202403-citibike-tripdata.zip!
All files were downloaded and packed out
Downloading 202404-citibike-tripdata.zip
Done with 202404-citibike-tripdata.zip!
All files were downloaded and packed out
Downloading 202405-citibike-tripdata.zip
Done with 202405-citibike-tripdata.zip!
All files were downloaded and packed out
Downloading 202406-citibike-tripdata.zip
Done with 202406-citibike-tripdata.zip!
All files were downloaded and packed out
```

```
Amount of files: 22

Files
202403-citibike-tripdata_1.csv - 186.31 MB

Total size: 0.18 GB
202401-citibike-tripdata_1.csv - 186.32 MB

Total size: 0.36 GB
202405-citibike-tripdata_1.csv - 186.58 MB

Total size: 0.55 GB
202403-citibike-tripdata_2.csv - 186.48 MB

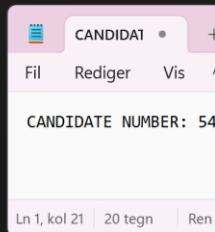
Total size: 0.73 GB
202405-citibike-tripdata_3.csv - 186.71 MB

Total size: 0.91 GB
202402-citibike-tripdata_3.csv - 22.56 MB

Total size: 0.93 GB
202405-citibike-tripdata_5.csv - 24.72 MB

Total size: 0.96 GB
202406-citibike-tripdata_5.csv - 146.46 MB
...
Total size: 3.24 GB
202402-citibike-tripdata_2.csv - 186.38 MB

Total size: 3.43 GB
```



Part B: Big Data Cleaning and Feature Engineering with PySpark

B1 – Loading Data

Initialized a SparkSession and all 22 CSV files were loaded into a single PySpark DataFrame and called `bike_df` using a wildcard path. This dataset included 13 columns with 18 807 481 rows before cleaning.

B2 – Schema and Nulls

The Schema was printed to confirm column types. I dropped some rows with null values in `start_station_name` or `end_station_name` with the help of `dropna()`.

18 758 555 rows remained after cleaning.


```

root
|-- ride_id: string (nullable = true)
|-- rideable_type: string (nullable = true)
|-- started_at: timestamp (nullable = true)
|-- ended_at: timestamp (nullable = true)
|-- start_station_name: string (nullable = true)
|-- start_station_id: string (nullable = true)
|-- end_station_name: string (nullable = true)
|-- end_station_id: string (nullable = true)
|-- start_lat: double (nullable = true)
|-- start_lng: double (nullable = true)
|-- end_lat: double (nullable = true)
|-- end_lng: double (nullable = true)
|-- member_casual: string (nullable = true)

rows after removing of nulls: 18758555

```

B3 – Feature engineering

Three new columns were created from the existing timestamp data. Subtracted `started_at` from `ended_at` and divided by 60 to convert seconds to minutes in order calculate `trip_duration_mins`. `Date_format()` with the `EEEE` pattern was used on extracting `day_of_week` to get full day names as Monday and Tuesday. To extract `hour_of_day` I used the `hour()` function. There were multiple ways I thought of to solve this question, but I decided to execute with this method.

```

+-----+-----+-----+-----+-----+
|      started_at      |      ended_at      | trip_duration_mins | day_of_week | hour_of_day |
+-----+-----+-----+-----+-----+
| 2024-01-01 13:00:... | 2024-01-01 13:04:... |          4.0       | Monday      | 13          |
| 2024-01-08 19:36:... | 2024-01-08 19:53:... |         16.55      | Monday      | 19          |
| 2024-01-12 15:00:... | 2024-01-12 15:36:... |          35.8       | Friday      | 15          |
| 2024-01-12 16:52:... | 2024-01-12 17:17:... | 25.166666666666668 | Friday      | 16          |
| 2024-01-05 19:50:... | 2024-01-05 20:34:... | 44.383333333333333 | Friday      | 19          |
+-----+-----+-----+-----+-----+
only showing top 5 rows

```

B4 – Data Filtering

Unusual trips were removed by keeping only rows where `trip_duration_mins` was strictly more than 1 minute and less than 1440 minutes. 18 758 357 rows remained after filtering.

rows after filtering: 18758357

CANDIDATE NUMBER: 54

Part C: Exploratory Data Analysis and Aggregation

C1 – User Behavior

Trips were grouped by `member_casual` to calculate the average trip duration and total trip count for each user type. The results show us that casual riders take significantly longer trips on average compared to members (20 minutes and 11 minutes). While member accounts for the majority of total trips with 15 326 682 trips vs 3 431 675 casual trips.

	member_casual	total_strips	avg_trip_duration_mins	File	Rediger	Vis	A
0	casual	3431675	20.101532	CANDIDATE NUMBER: 54			
1	member	15326682	11.322308				

C2 – Top Routes

I identified the 5 most frequent routes by grouping on `start_station_name` and `end_station_name`. What I noted was that the three most popular routes are round trips where the start and end station are the same. This shows that many riders return their bikes to their starting point.

	start_station_name	end_station_name	count	CANDIDAT	File	Rediger	Vis	A
0	Central Park S & 6 Ave	Central Park S & 6 Ave	6748	CANDIDATE NUMBER: 54				
1	7 Ave & Central Park South	7 Ave & Central Park South	5381					
2	Roosevelt Island Tramway	Roosevelt Island Tramway	4159					
3	River Ter & Warren St	Vesey Pl & River Terrace	3378					
4	Grand Army Plaza & Central Park S	Grand Army Plaza & Central Park S	3236					

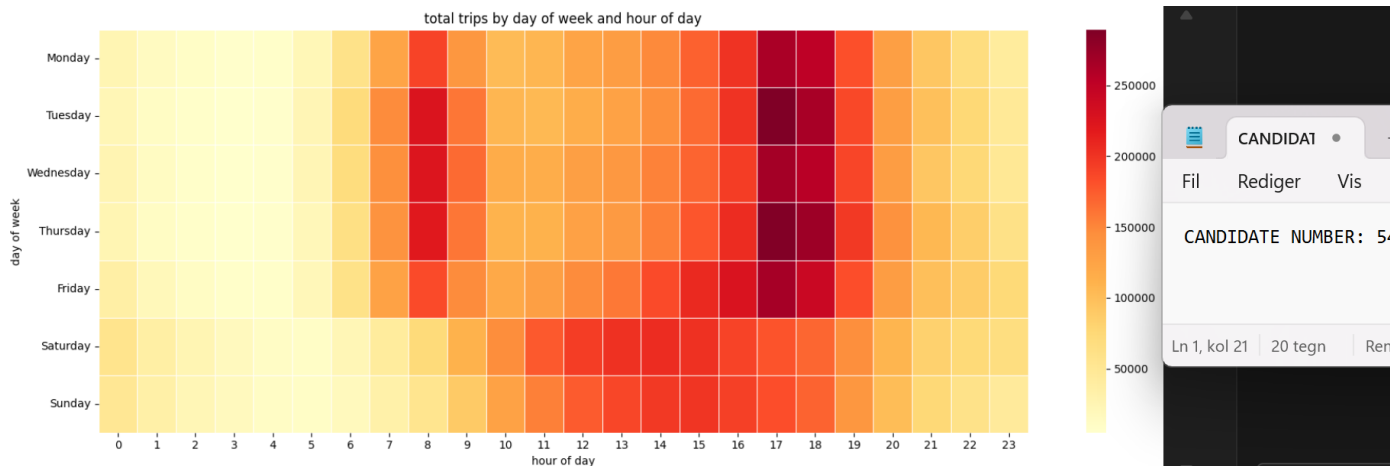
C3 – Save Aggregation

The user behavior aggregated DataFrame was saved to a SQLite database with the name of `citibike_summary.db`. I did this with the help of using `to_sql()` as a backup for further analysis.

Part D: Advanced Data Visualization

D1 – Time Distribution (Heatmap)

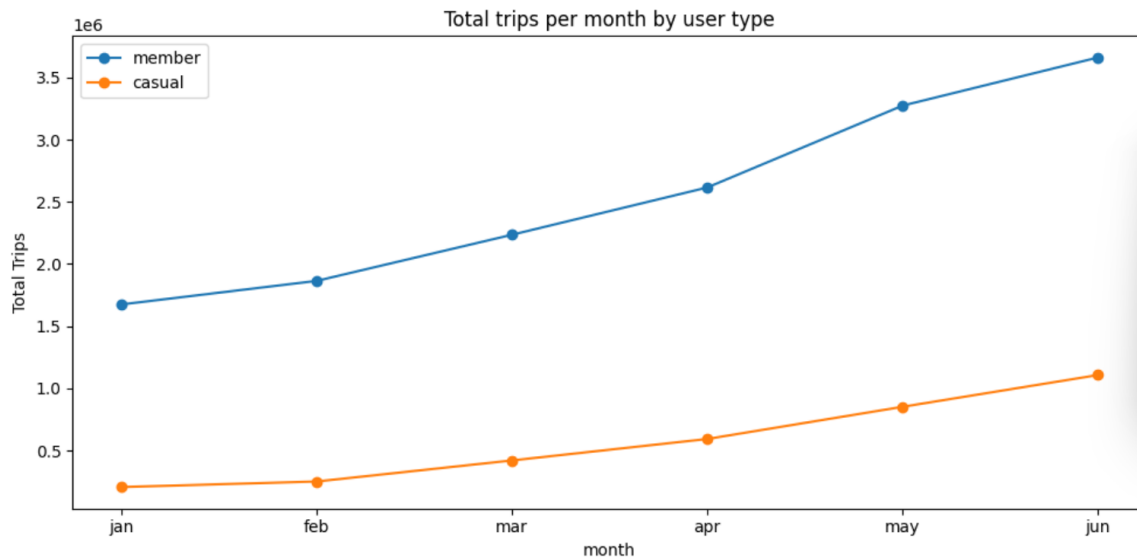
I created a heatmap that shows total trips by day of week on the Y-axis and hour of day on the X-axis. All days are ordered from Monday to Sunday. The visualization shows two daily rush hour peaks on weekdays around 8 am and 5-6 pm (17-18 CET on the heatmap), while weekends show more evenly distributed activities throughout the day.



This pattern tells us that members primarily use the bike-sharing service for commuting purposes, while casual riders tend to use it more for leisure, mostly during weekend afternoons.

D2 – Monthly Trend (Line Chart)

I created a line chart that shows total trips per month from January through June, with separate lines for members and casual riders. What I noted here was that both user types show a consistent upward trend as the temperatures rise through spring and early summer, with June being peak month for both groups. As expected, members consistently outnumber casual riders across all months.



This is exactly what I expected and how I would use this service if I was a member of casual, as warmer weather encourages outdoor activities. Still the gap between member and casual riders remains relatively stable across all months, which suggests that the weather affects both groups equally.

Conclusion

This analysis tells us how PySpark can efficiently process large amounts of real-world datasets that could be impractical to handle with traditional tools. The citi bike data reveals clear behavioral differences between members and casual riders, where the members use the service primarily for commuting and casual riders for leisure. The seasonal trend confirms that bike-sharing activities are strongly influenced by weather conditions.

Question 3: Global Earthquake Tracker App Using Streamlit

Introduction

In this task I built an interactive web application with the help of Streamlit. Streamlit lets the users explore recent global earthquake activity. This data comes from the United States Geological Survey, which is a public API. It provides real-time seismic data from the past 30 days. The target here was to make the data easy to explore through filters and a live map.

Part A: Data Extraction and Preprocessing

I sent a GET request to the USGS API endpoint with the help of requests library in order to fetch the earthquake data. The response I got back was as a GeoJSON file, so I had to loop through the features array to pull out the relevant fields. For every earthquake I extracted the place, magnitude, time, longitude, latitude and depth. I stored everything in a list of dictionaries right before converting it to a pandas DataFrame that was called `quake_df`.

I stumbled upon a problem, the time field came in as Unix milliseconds. That's when I tried to convert it to a proper datetime format using `pd.to_datetime()` with `unit='ms'`. After that I dropped any rows that had missing values in the magnitude or coordinate columns to make sure that the data was all clean before doing anything more.

After I cleaned the data, I created a new column that was called `risk_category`. It was based on the magnitude of each earthquake. All earthquakes below 3.0 was labeled Minor, while those between 3.0 and 4.9 were labeled Moderate. And finally everything with 5.0 or above that was labeled Strong.

... 2026-06-02 18:10:11.987 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thre

	Place	Magnitude	Time \
0	9 km N of Borrego Springs, CA	1.75	2026-06-02 16:04:32.030
1	1 km N of The Geysers, CA	0.77	2026-06-02 16:03:00.460
2	34 km SE of Malaga, New Mexico	1.50	2026-06-02 15:57:52.251
3	10 km SSW of Idyllwild, CA	0.72	2026-06-02 15:55:42.000
4	3 km ENE of Healdsburg, CA	0.93	2026-06-02 15:27:02.870
5	6 km SSW of Redlands, CA	0.74	2026-06-02 15:19:30.910
6	2 km NNW of The Geysers, CA	0.75	2026-06-02 15:16:34.980
7	5 km NW of The Geysers, CA	0.74	2026-06-02 15:09:49.480
8	1 km SSE of Talkeetna, Alaska	1.50	2026-06-02 15:06:30.802
9	2 km ESE of Tyonek, Alaska	1.90	2026-06-02 15:00:43.017
10	17 km WSW of Johannesburg, CA	0.92	2026-06-02 14:54:03.880
11	6 km WNW of The Geysers, CA	0.77	2026-06-02 14:42:32.690
12	7 km N of Naalehu, Hawaii	1.84	2026-06-02 14:25:49.640
13	4 km NNW of The Geysers, CA	1.05	2026-06-02 14:14:48.990
14	7 km ESE of Big Spring, Texas	1.60	2026-06-02 14:07:10.430
15	20 km ESE of Julian, CA	0.82	2026-06-02 14:00:37.580
16	232 km E of Levuka, Fiji	4.60	2026-06-02 13:57:47.568
17	4 km SE of San Jacinto, CA	0.66	2026-06-02 13:57:13.450
18	11 km N of Susitna North, Alaska	2.00	2026-06-02 13:53:59.909
19	0 km SSE of Huron, CA	2.64	2026-06-02 13:48:00.420

	Longitude	Latitude	Depth	Risk Level
0	-116.393500	33.336500	6.9000	Minor
1	-122.757332	38.784000	3.6700	Minor
...				
17	-116.929167	33.762333	13.9100	Minor
18	-149.873000	62.266000	0.1000	Minor
19	-120.099998	36.199001	7.9300	Minor
(10309, 7)				

The final dataset consists of 10 309 rows with 7 columns after the cleaning.

Part B: Streamlit App Development

I was wondering what name to use for the title but ended up setting the page title to “Global Earthquake Tracker”. I used a wide layout to make better use of the screen space. I also added a section below that explains what the app does and how to use the filters. I chose to make the app really comfortable for the user by keeping all the user controls on the sidebar so that the main area stays clean and focused on the data.

I added two sliders in the sidebar with the help of using `st.sidebar.slider()`. With that you can filter the minimum magnitude and the maximum depth. When adjusting the sliders, the map will automatically update to the user interface.

I also added a multiselect widget with the help of using `st.sidebar.multiselect()` that lets the users filter by Minor, Moderate and Strong earthquakes. All of them are chosen by default. It is the same as the sliders, if you decide to remove one of them or only keep one

or two of them in the widget, the app will automatically update, and show based on what you filter on.

Over the map I used `st.columns()` to display two metrics side by side. The first shows total earthquakes based on what the user filters on, while the second shows maximum magnitude based on the same reason.

With the help of `st.map()` I could plot all filtered earthquake locations on an interactive world map. With the same principle as the other implementations, the map updates based on user interface.

